

-----::CulinaryCart Project Report::-----

Introduction

CulinaryCart is a food ordering and cart management system built with ASP.NET Core, C#, and Entity Framework Core. It enables users to browse menu items, apply offers, manage their cart, and complete checkout. The system is designed with a layered architecture to ensure scalability, maintainability, and clear separation of concerns.

- Provide a structured way to manage food menu items.
- Allow cart operations (add, update, delete).
- Apply business rules for offers and discounts.
- Maintain order history for checkout.
- Ensure clean separation between **Controllers**, **BAL**, **DAL**, and **DbContext**.

System Architecture

The project follows a **4-layer architecture**:

- **Controllers**: Handle HTTP requests and responses.
- **BAL (Business Access Layer)**: Contains business rules (offers, totals, cart logic).
- **DAL (Data Access Layer)**: Handles database CRUD operations.
- **DbContext**: Entity Framework Core context for database communication.
- **Models**: Define entities like Menu and OrderHistory.

Key Components

- **MenuDAL** → Provides menu items from the database.
- **CartBAL** → Applies offers, calculates final prices, manages cart operations.
- **OrderHistoryDAL** → Stores cart and checkout history.
- **CulinaryCartDbContext** → Manages database tables.

Dependencies and NuGet Packages

- **Microsoft.EntityFrameworkCore**
- **Microsoft.EntityFrameworkCore.SqlServer**
- **Microsoft.EntityFrameworkCore.Tools**
- **Microsoft.AspNetCore.Mvc**
- **Microsoft.AspNetCore.Http**
- **System.Linq**
- **System.Collections.Generic**

Advantages

- **Separation of concerns:** BAL handles rules, DAL handles data.
- **Scalability:** Easy to add new offers or features.
- **Maintainability:** Clean layered design.

Future Enhancements

- Role-based access (Admin vs Customer).
- Payment gateway integration.
- Order tracking and notifications.
- JWT authentication for secure sessions.